

0. 論理設計と物理設計はトレードオフの関係にある
→ 理解した上で、望ましい判断を述べる必要がある

1. データベースのモデル
- ・ リレーショナルデータベース(RDB) ... 二次元表形式での管理
 - ・ オブジェクト指向データベース(ODB) ... オブジェクトの持つ形式のまま保存
 - ・ XMLデータベース
 - ・ キーバリュ型ストア(KVS) ... 識別キーと値をセットで保存
 - ・ ドキュメント型データベース ... JSON形式で保存するDBなど
 - ・ 階層型データベース ... 木構造で表現
- } NoSQL
- オープンソース(無料)

主要なDBMSには、Oracle, SQL Server, Db2, Postgres, MySQL などがある

開発手法 { ウォーターフォールモデル ... 要件定義 → 設計 → 実装 → テスト の流れで、ステップごとに着実に進む
アジャイルモデル ... 上記のサイクルを何度も繰り返す (1サイクルをスプリントという)

トピック { データ中心アプローチ(DOA) ... データ設計 → アプリケーション設計
プロセス中心アプローチ(POA) ... 上記の逆

DB設計は一度頼み込んでおけばいいので、DOAの方が理にかなっている

3層スキーム { 外部スキーム(ユーザ視点) ... データ独立性を担保するため必要(外部と内部の分離) 物理層の制約に依存しない
概念スキーム(開発者) ... データ同士の関係性を記述。この設計を論理設計という
内部スキーム(DBMS) ... インデックスなどの物理的定義を記述。物理設計

2. 論理設計のステップ (物理層にはとらわれない)

1. エンティティの抽出 ... どのようなデータが必要なのかを洗い出す (C 要件定義)
2. " の定義 ... " 属性(カラム) "
3. 正規化 ... エンティティのフォーマットを整理すること
4. ER図の作成 ... IF記法やIDEFIXがスタンダード

物理設計のステップ

1. テーブル定義 ... ER図のようなレベルのモデル(論理モデル)から、物理モデルを作成する
2. インデックス定義 ... パフォーマンス向上のための索引をつくる
3. ハードウェアのサイジング ... キャパシティとパフォーマンスの2観点でのサイジング
ストレージのI/OがDBのパフォーマンスを左右する
正規化による整合性とパフォーマンスには、トレードオフの関係がある
⇒ スケーラビリティの高さ、構成を柔軟にすることが重要
4. ストレージ冗長構成 ... RAID(ディスクの冗長配列)によりデータを分散し、信頼性と性能を高める
耐障害性をもたせることは、絶対条件
データの更新を一時的に保持
トランザクションの整合性を保つ
5. ファイルの物理配置 ... テーブルファイル、インデックスファイル、システムファイル、一時ファイル、ログファイルの配置決定
現用系は、
→ サプリメントなどの避難場所

レプリケーション ... トランザクションログの更新情報をコピーして、待機系を作成すること
コピーのタイミング次第で、同期・非同期レプリケーションに分かれる
読み取り専用のリードレプリカを用いることで負荷分散も可能
ログを転送する物理レプリケーション、SQLを転送する論理レプリケーションがある

クラウド冗長構成 ... AZ内にデータをレプリケーションし、分散配置を実現(RAIDとは異なる)
DBのアップグレード(バックアップ)とリリージョンに保存しておくことも有効

バックアップコストとリカバリコストは二律背反

バックアップ方式 (とやかくのみとやかくはなく、相対合わせることが一般的)

- フルバックアップ ... テーブルや設定ファイル、トランザクションログを全バックアップして保存
時間的・経済的コストが大きい
 - 差分バックアップ ... フルバックアップと直近のとやかくの差分のトランザクションログをバックアップして保存
 - 増分バックアップ ... 直前のバックアップとの差分をバックアップして保存
とデータ量が減る代わりに、リカバリ時間が増え、完全復旧の可能性は低いとされている
- 永続増分バックアップなどもある

リカバリ設計

1. フルバックアップのテーブルとDBに保存 (リストア)
2. 差分・増分バックアップのログとテーブルとDBに保存 (リカバリ)
3. DBMS内に残ったトランザクションログを適用する (ロールフォワード)

3. 「テーブルとは、すばらしい複数形 or 集合名詞 であること」 by J. Celko

キー ... テーブルを引くための鍵のようなもの

・主キー ... テーブルを一対一に識別するに必要で、可変長文字列のカラムをキーにすることはアンチパターン

・外部キー ... 外部テーブルを参照するキーで、参照整合性を制約を課す

親テーブルの削除や更新時に、子テーブル側で処理することと「カスケード」

正規化 ... テーブルの冗長性や非一貫性の問題を解決するための方法

・第1正規形 ... 1つのセルの中に1つの値しか含まない (セルにはスカラ値のみ)

概念としてキーを決めれば、値が決まるという原則に反するため

⇒ テーブルを適切に分割する

・第2正規形 ... 部分関数従属を解消し、完全関数従属のみでテーブルを作ること

主キーに部分依存したデータは追加できない、本来同じデータが入るものと登録できる可能性のため

⇒

・第3正規形 ... 推移的関数従属を解消する

特定のカラムのみには依存したデータは追加できない

⇒

・ボイス・コッド正規形 ... 非キーからキーへの関数従属をなくした状態にすること

冗長な更新や、カスケードによる想定外の挙動が起こる可能性があるため

⇒ (非可逆分割にならないように注意)

・第4正規形 ... エンティティ同士の関連を表現する関連エンティティを作る場合、その関連は1対1にすること

キーに対する多値従属性を解消する

主キーがNULLのデータの登録や、更新の冗長性を防ぐ

⇒

・第5正規形 ... 関連と関連エンティティを1対1対応させる

⇒

4. ER図 { ER記法 (カーズ+リティ 0=0, =1, 1=1, 1=1, 1=1)
IDEFIX

カーズ+リティ ... 相手のエンティティと対応するコード数

関連実体 ... 多対多の関連を表現するためのエンティティで、いわゆる中間テーブルのこと

5. 正規化と検索SQLのパフォーマンストレードオフの関係にある
- 正規系 ... テーブルの整合性を保ちやすく、更新処理が高速
 - 非正規系 ... 結合の処理が速くなるが、テーブルの取得処理が高速 (最終手段という態度が適切)
 - パフォーマンス向上と目的は異なる非正規化
 - ・ サマリテーブルの保持
 - ・ 選択条件に使用するテーブルの保持
 ⇒ 結合による負荷を回避することになる
 - ※ 更新時のパフォーマンスやリアルタイム性とのトレードオフは慎重に検討する必要あり

6. B-tree インデックス ... 非ヒューマンインデックスで、木構造でデータを保持する
- ・ 均一性 ... 平衡木であるため、探索と均一な量で行うことができる
 - ・ 持続性 ... データ量に対して、性能が緩やかに変化する (性能はデータ量 n に対して $O(\log n)$)
 - ・ 処理効率性 ... 挿入、更新、削除処理も $O(\log n)$
 - ・ 非等値性 ... 不等号も同様に高速で使用できる。ただし否定条件は例外
 - ・ 親ソート性 ... 暗黙的にソートされているため、明示的なソートは不要 ⇒ インデックスカラムのソートは軽い設計方針

- ・ データ量の多いテーブルに実装する (目安として、数万以下のデータ量は、インデックスが効果を発揮しない)
- ・ カージナル性が高く、分散の大きいカラムに実装する (目安として、残り読み率 5% 以下、集中傾向)
- ・ 裸の値、検索条件に使用する (インデックス列に演算、IS NULL、否定形、後方一致、中間一致、暗黙的な型変換を行かせると、インデックスが使用されない)
- ※ 主キー一意制約のデータは自動的にインデックスが作成される
- ※ 更新性能が悪化するのを、禁止 (インデックスシャットダウンというオプション)
- ※ インデックスを再構築するなどのメンテナンスは必要

- SQLを送信したら実行されるまでの流れ
1. パーサ ... SQLが適切なかどうかをチェック
 2. オプティマイザ ... 実行計画を決定。このとき、カタログメタデータの統計情報を参照して、経路選択
 3. テーブルアクセス
- ※ 統計情報の質が最適化の質につながる ⇒ 都度更新するべき (大量のデータ更新時には夜間など)
- 上記以外のパフォーマンスチューニング

- ・ パーティション ... 分割し、キーで、データの物理配置をキーでまとめる (パーティションキー)
 - 物理的なデータ分割 WHERE 句条件での指定が効率的にあり、データの読み込み量が減る
- ・ ヒント句 ... SQLに実行計画を立てるためのヒントを与えることが可能 (最終手段)
- ・ パラレル化 ... マルチコアでSQLを実行。逆効果になるケースもあるので注意が必要

7. 論理設計のオプション
- ・ 非スカラ値 ... 配列型などのサポートもあるが、第1正規形であることを優先すべき
 - スカラ値は、意味的に分割できる限り、なるべく分割して保存すべき
 - ・ データリネーミング ... カラムの意味を一度決めたら、途中で変更はしない
 - ・ 単一参照テーブル ... テーブル構造の等しい、別々のエンティティを1つのテーブルにまとめること (ポリモルフィズムは不要)
 - ・ テーブル分割 ...
 - 水平分割 ... 意味的な分割の理由がなく、パフォーマンス面では代替がある (パーティション等)
 - 垂直分割 ... テーブルマート (列の読み込みテーブル) やサマリテーブルなどの代替がある
 - ↳ DWHが切り出しの小規模テーブル

- ・不適切なキー ... 固定長を使用し、外部キーなどと同じ意味のテーブル型は等しくすべき
- ・アトリビュート ... 同じ役割を果たすテーブルが2つ以上ある状態は、システム統合時に起こりうる可能性がある
テーブルレングス ... 行う必要がある
- ・バッチモード ... 使用すべきに放置すべきはテーブルモード。ストレージの容量を食うため、コストがかかる
- ・多段モード ... ネストしたテーブルモード。テーブルと連った、1つ時点のテーブル数関係は不明になる
⇒ テーブルモードには、設計書必須

8. 論理設計のルール

- ・代理キー ... キーがハイリミットに利用すべき (使用回) ... 途中が指す内容が変化する
 ⇒ サロゲートキーという工学的なキーがあるオートインクリメントを使用する (ルール17(17))
このルールは理解し難い
否定の理由も
前提がわからない
 ⇒ カラムと意味のあるテーブルの構成要素自然キーを用いると良い
 - ・リッチテーブル ... 仕様変更が弱く、無用のNULLを使わざるを得なくなる (スリッパカラムアトリビュート)
 ⇒ 行持ちテーブルと利用する (= 別テーブルに分離)
 - ・アドホックな集計キー ... 場当たり的な集計用カラムは、100%コンプライアンスに合わせる
 ⇒ SQLでエミュレートし、ビューを使用し、その材料になる
 - ・多段ビュー ... ビューはあくまで、リテラを保存するキータブル (リテラの生話)
 ⇒ ユーザ → ビュー → テーブルの構成になっている。ビューがネストすると基盤テーブルが隠れる
 テーブルレングス ... テーブルを整理すると、テーブルバース設計よりも前に行う
 一意なキーを特定することが重要
 DBの統合時には必要に応じて、テーブルの重複を防ぎ、一元管理できるようにする
- NewSQL ... Cloud Spanner, CockroachDB など

- ## 9. 隣接リストモデル
- ポインタのような自己参照をすることで、木構造を表現するモデル
 大規模な木構造でも使えど不明だが、再帰共通形式により、効率的に処理できる
 更新や挿入も手軽に行うことができる
- 閉包テーブルモデル ... 2つのテーブルを使い、特徴的な方は、もう片方のテーブルのidとそちらに当たるテーブルのidを持つ
 カウントなどでは、反面、更新処理は整合性を保つために丁寧に行う必要あり